Lista 11

Marta Hałas

2025-01-26

Spis treści

1 sekcja 1 **1**
1.1 zadanie 1 1
1.2 zadanie 2 3

2 sekcja 2 **4**
2.1 zadanie 1 4
2.2 zadanie 2 5
2.3 zadanie 3 6
2.4 zadanie 4 8

3 sekcja 3 **9**
3.1 zadanie 1 9
3.2 zadanie 2 16

1 sekcja 1

1.1 zadanie 1

a)

```
n<-10
N<-1000
symulacja1<- function(n) {
  urodziny<- sample(1:365, n, replace=TRUE)
  if (any(duplicated(urodziny))) {
    return (TRUE)
  } else {
    return (FALSE)
  }
}

symulacja<- function(n,N) {
```

```r
  a <- sapply(rep(n,N), FUN = symulacja1)
  return(sum(a)/N)}

symulacja(n,N)
```

```
## [1] 0.126
```

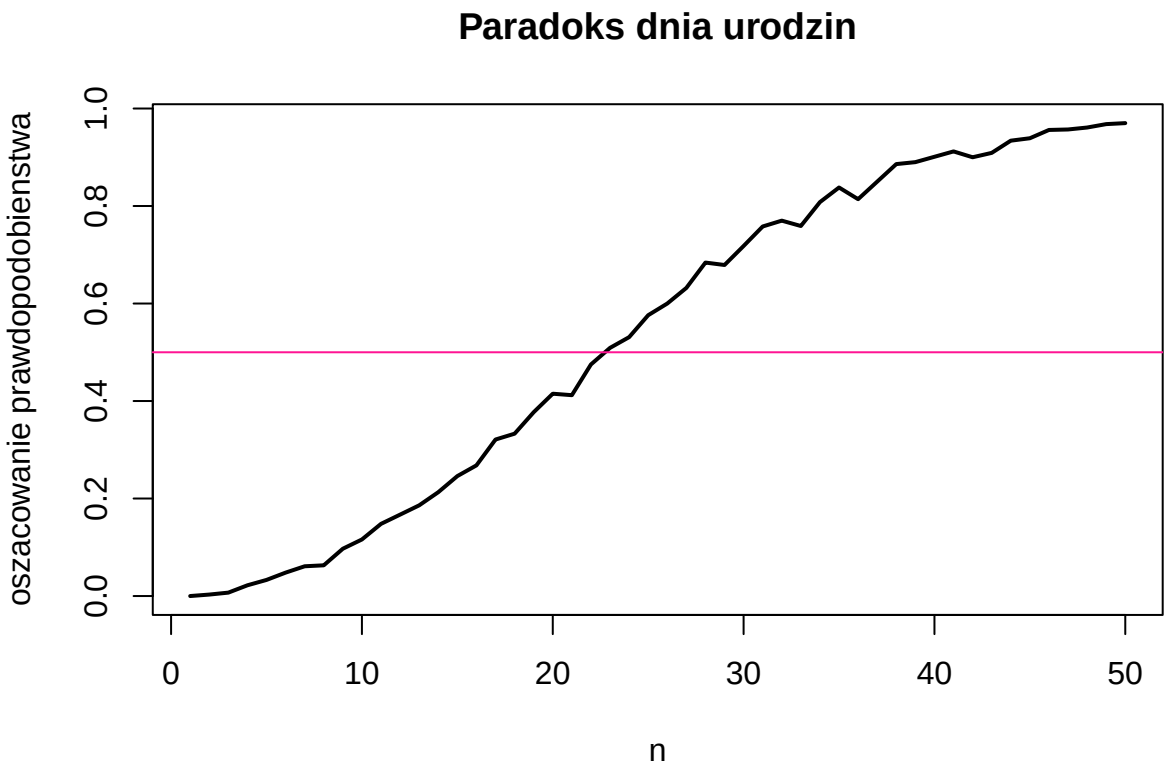
```r
symulacja(600,N)
```

```
## [1] 1
```

Korzystając z zasady szufladkowej dirichleta dla n=600 powinno wyjść P=1.

b)

```r
x<-seq(1,50)
y <- sapply(x,FUN = symulacja, N)
plot(x,y, type="l", lwd=2, xlab="n", ylab="oszacowanie prawdopodobieństwa")
title("Paradoks dnia urodzin")
abline(h=0.5, col="deeppink")
```




```r
symulacja(20,N)
```

```
## [1] 0.424
```

```r
symulacja(25,N)
```

```
## [1] 0.579
```

Grupa powinna zawierać co najmniej 25 osób.

1.2 zadanie 2

```r
n<-10000
symulacja<- function(n) {
wyniki <-numeric(n)
for (i in 1:n) {
  orzeł<-sum(sample(c(0,1), size=3, replace = TRUE))
  if (orzeł==0) {
  wyniki[i]<-0
  } else {
  wyniki[i]<-sum(sample(1:6, size=orzeł, replace = TRUE))
    }
}
  return (wyniki)
}
wyniki<-symulacja(n)
czestosc<-table(wyniki)/n
czestosc
```

```
## wyniki
##      0      1      2      3      4      5      6      7      8      9     10
## 0.1254 0.0587 0.0723 0.0836 0.0931 0.1084 0.1238 0.0699 0.0638 0.0577 0.0487
##     11     12     13     14     15     16     17     18
## 0.0370 0.0250 0.0122 0.0082 0.0070 0.0031 0.0012 0.0009
```

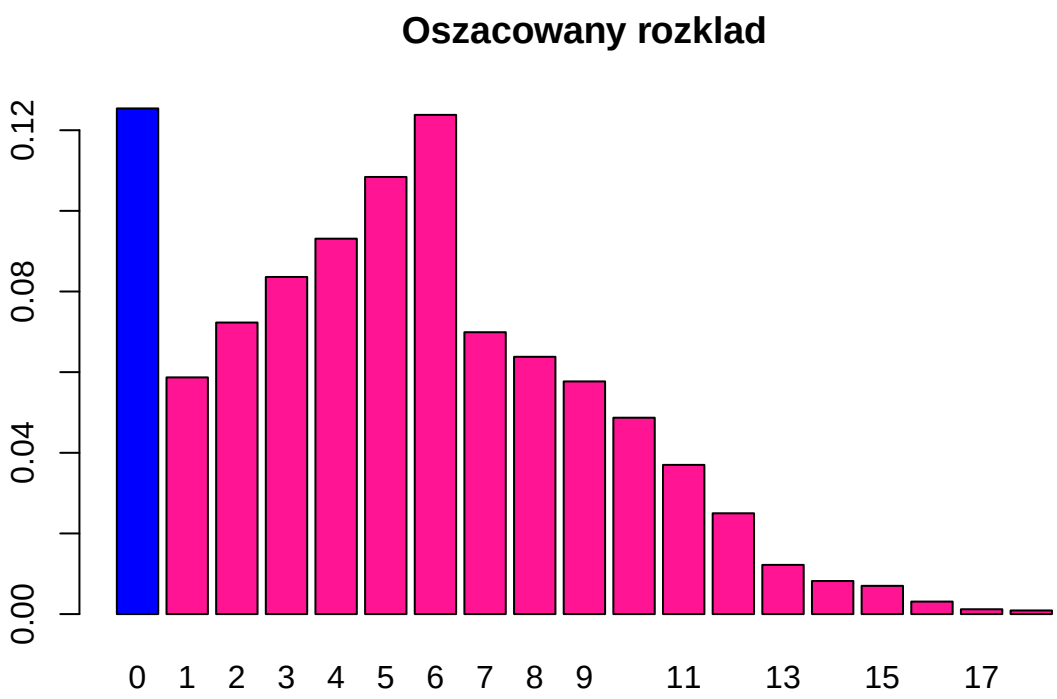
```r
wartosc_oczekiwana <- mean(wyniki)
wartosc_oczekiwana
```

```
## [1] 5.2889
```

```r
id.max<-which(czestosc==max(czestosc))
czestosc[id.max]
```

```
##      0
## 0.1254
```

```r
kolory.slupkow<-rep("deeppink", length(czestosc))
kolory.slupkow[id.max]<-"blue"
barplot(czestosc, main="Oszacowany rozkład", col=kolory.slupkow)
```



2 sekcja 2

2.1 zadanie 1

a)

i)

```
dbinom(x=0, size=200, prob=0.01)
```

```
## [1] 0.1339797
```

ii)

```
sum(dbinom(x=seq(0,5,1), size=200, prob=0.01))
```

```
## [1] 0.9839771
```

iii)

```
1-sum(dbinom(x=c(0,2,1), size=200, prob=0.01))
```

```
## [1] 0.3233213
```

b)

```r
dpois(x=0,lambda=200*0.01)
```

```
## [1] 0.1353353
```

```r
sum(dpois(x=seq(0,5,1),lambda=200*0.01))
```

```
## [1] 0.9834364
```

```r
1-sum(dpois(x=c(0,2,1),lambda=200*0.01))
```

```
## [1] 0.3233236
```

Pierwsze dwie liczby po przecinku są sobie równe dla tych dówch rozkładów.

2.2 zadanie 2

a)

```r
k<-c(1,2,3)
normalny<- function (k) (pnorm(k)-pnorm(-k))
p<-sapply(k, FUN=normalny)
p
```

```
## [1] 0.6826895 0.9544997 0.9973002
```

```r
mean=2
sd=3
normalny<- function (k) (pnorm(mean + sd*k,mean,sd)-pnorm(mean-k*sd,mean,sd))
p<-sapply(k, FUN=normalny)
p
```

```
## [1] 0.6826895 0.9544997 0.9973002
```

Nie, wyniki są takie same, niezależnie od parametru jaki został przyjęty.

b)

```r
symulacja<-function (n, ś, s, k) {
  próbki<- rnorm(n, mean=ś, sd=s)
  filtrowanie<-function(k) {
    a<-próbki[próbki >= (ś - k * s) & próbki <= (ś + k * s)]
    PA <- length(a)
    return (PA/length(próbki))}
  return (sapply(k, FUN=filtrowanie))

}
n<-seq(1,1000,200)
k<-c(1,2,3)
normalny<- function (k) (pnorm(k)-pnorm(-k))
p<-sapply(k, FUN=normalny)
p
```

```
## [1] 0.6826895 0.9544997 0.9973002
```

```r
sapply(n,FUN = function(n) symulacja(n,0,1,k))
```

```
##      [,1]      [,2]      [,3]      [,4]      [,5]
## [1,]    0 0.7164179 0.6558603 0.6971714 0.7153558
## [2,]    1 0.9452736 0.9326683 0.9550749 0.9575531
## [3,]    1 1.0000000 0.9975062 0.9966722 0.9962547
```

```r
mean=2
sd=3
normalny<- function (k) (pnorm(mean + sd*k,mean,sd)-pnorm(mean-k*sd,mean,sd))
p<-sapply(k, FUN=normalny)
p
```

```
## [1] 0.6826895 0.9544997 0.9973002
```

```r
sapply(n,FUN = function(n) symulacja(n,2,3,k))
```

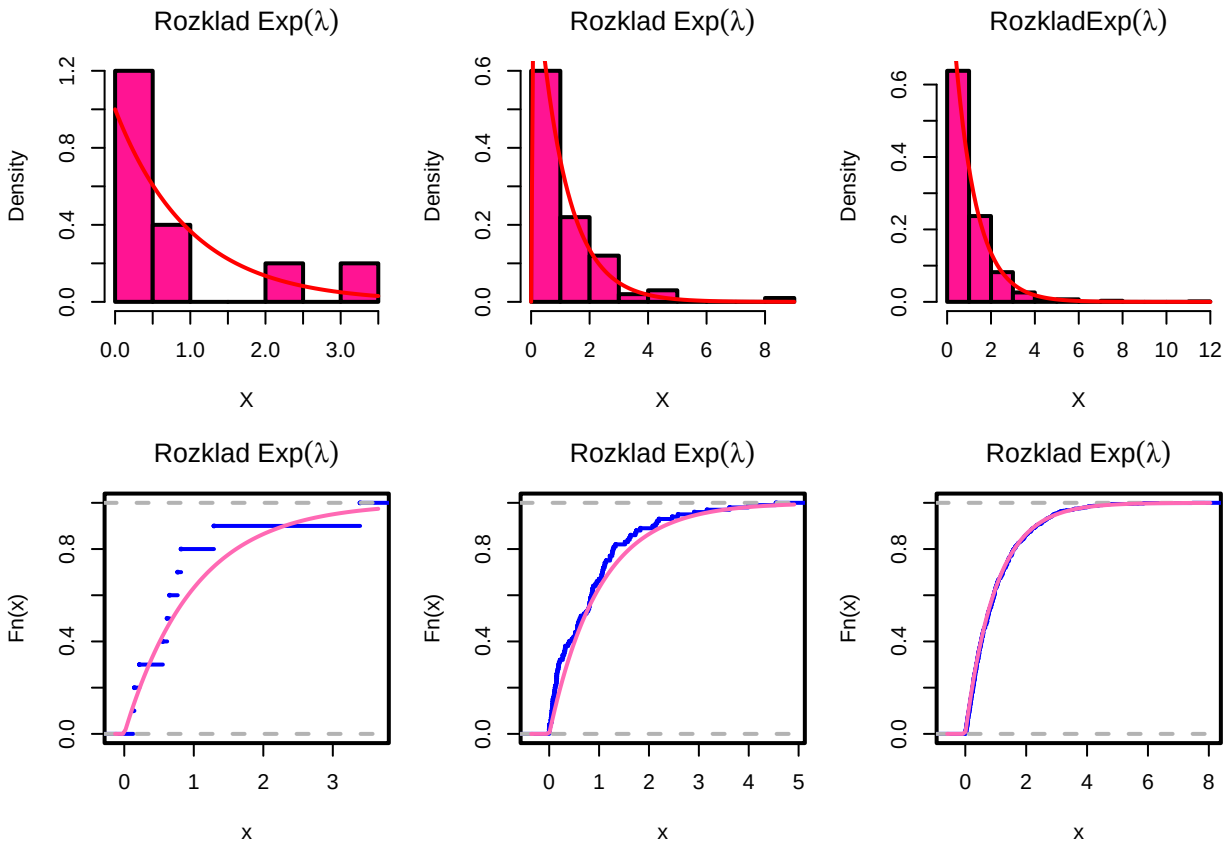
```
##      [,1]      [,2]      [,3]      [,4]      [,5]
## [1,]    1 0.6716418 0.6857855 0.7004992 0.6704120
## [2,]    1 0.9552239 0.9301746 0.9683860 0.9538077
## [3,]    1 0.9900498 0.9950125 0.9983361 0.9987516
```

Dla n=601 oraz n=801 otrzymujemy zadowalające przybliżenie.

2.3 zadanie 3

```r
par(mfrow=c(2,3))
old.mar <- par("mar")
par(lwd=2, mar=c(4.1, 4.1, 3.1, 1.1))

my.rexp <- function(n, lambda=1)
{
  U <-  runif(n)
  X <- -log(1-U)/lambda
  return(X)
}
lambda<-1
n<-10
X<- my.rexp(n=n, lambda = lambda)
hist(X, prob=TRUE, col="deeppink", main="")
curve(dexp(x,rate=lambda),col="red",add=T, lwd=2)
title(expression(paste('Rozkład ', Exp(lambda))))


n<-100
X<- my.rexp(n=n, lambda = lambda)
hist(X, prob=TRUE, col="deeppink", main="")
```

```r
curve(dexp(x,rate=lambda),col="red",add=T, lwd=2)
title(expression(paste('Rozkład ', Exp(lambda))))


n<-1000
X<- my.rexp(n=n, lambda = lambda)
hist(X, prob=TRUE, col="deeppink", main="")
curve(dexp(x,rate=lambda),col="red",add=T, lwd=2)
title(expression(paste('Rozkład', Exp(lambda))))


n<-10
X<- my.rexp(n=n, lambda = lambda)
plot(ecdf(X), lwd=2, cex=.1, main="", col="blue")
curve(pexp(x, rate=lambda), lwd=2, col="hotpink", add=T)
title(expression(paste('Rozkład ', Exp(lambda))))


n<-100
X<- my.rexp(n=n, lambda = lambda)
plot(ecdf(X), lwd=2, cex=.1, main="", col="blue")
curve(pexp(x, rate=lambda), lwd=2, col="hotpink", add=T)
title(expression(paste('Rozkład ', Exp(lambda))))


n<-1000
X<- my.rexp(n=n, lambda = lambda)
plot(ecdf(X), lwd=2, cex=.1, main="", col="blue")
curve(pexp(x, rate=lambda), lwd=2, col="hotpink", add=T)
title(expression(paste('Rozkład ', Exp(lambda))))
```



Tak, przypominają swoje teoretyczne odpowiedniki dla n=100.

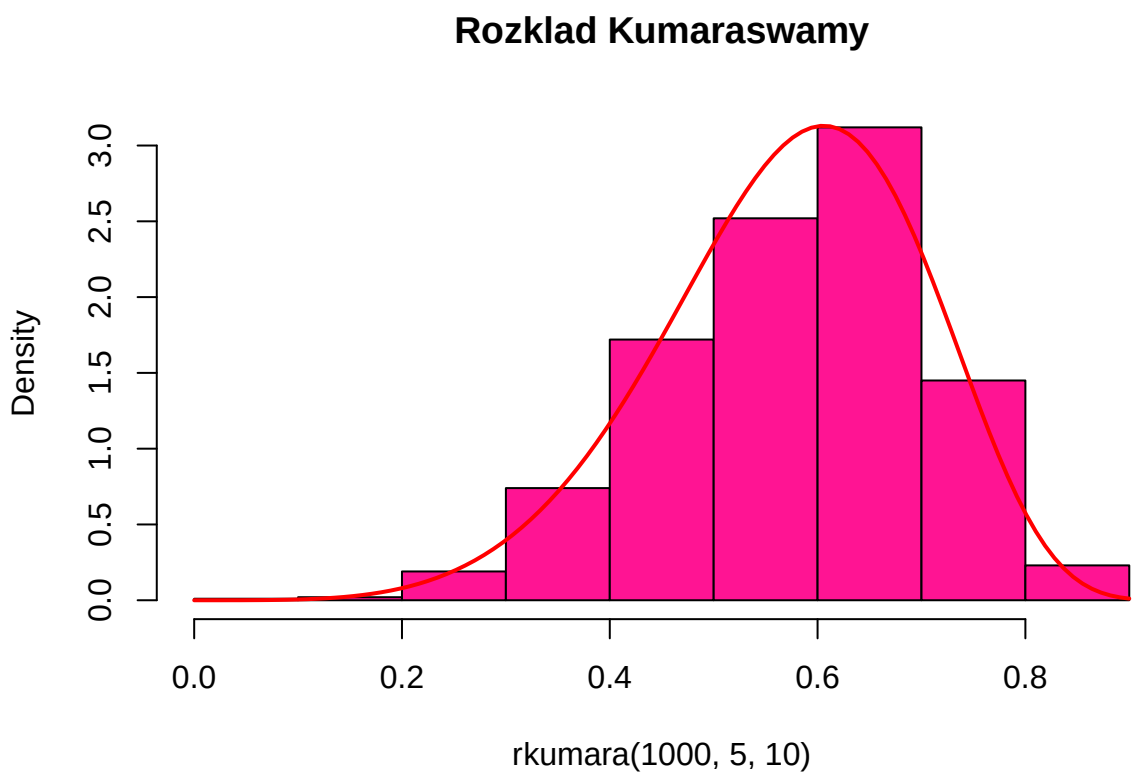
2.4 zadanie 4

a)

```
rkumara<- function(n,a,b) {
  y<-runif(n, 0,1)
  return ((1-(1-y)^(1/b))^(1/a))
}
```

b)

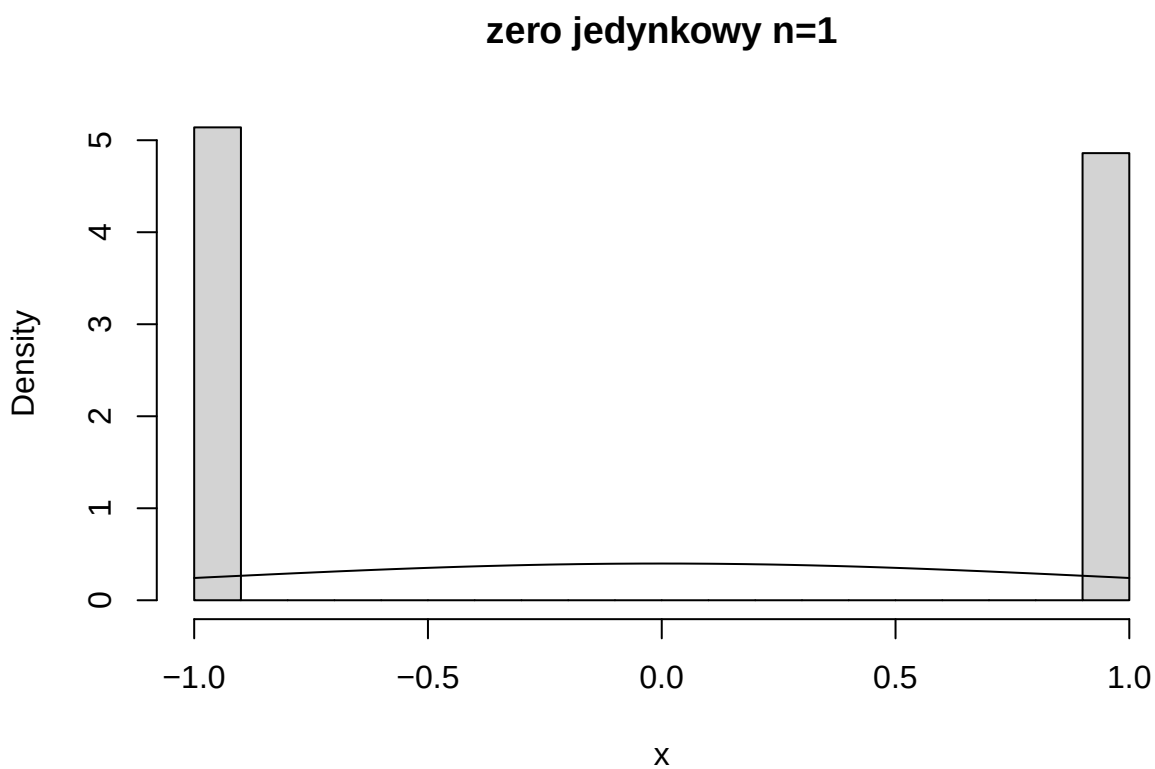
```
grkumara<- function(x,a,b) {
 ifelse( x>=0 & x<=1, a*b*(x^(a-1))*(1-x^(a))^(b-1),0)
}
hist(rkumara(1000,5,10), prob=TRUE, col="deeppink", main="")
curve(grkumara(x,5,10),col="red",add=T, lwd=2)
title('Rozkład Kumaraswamy')
```



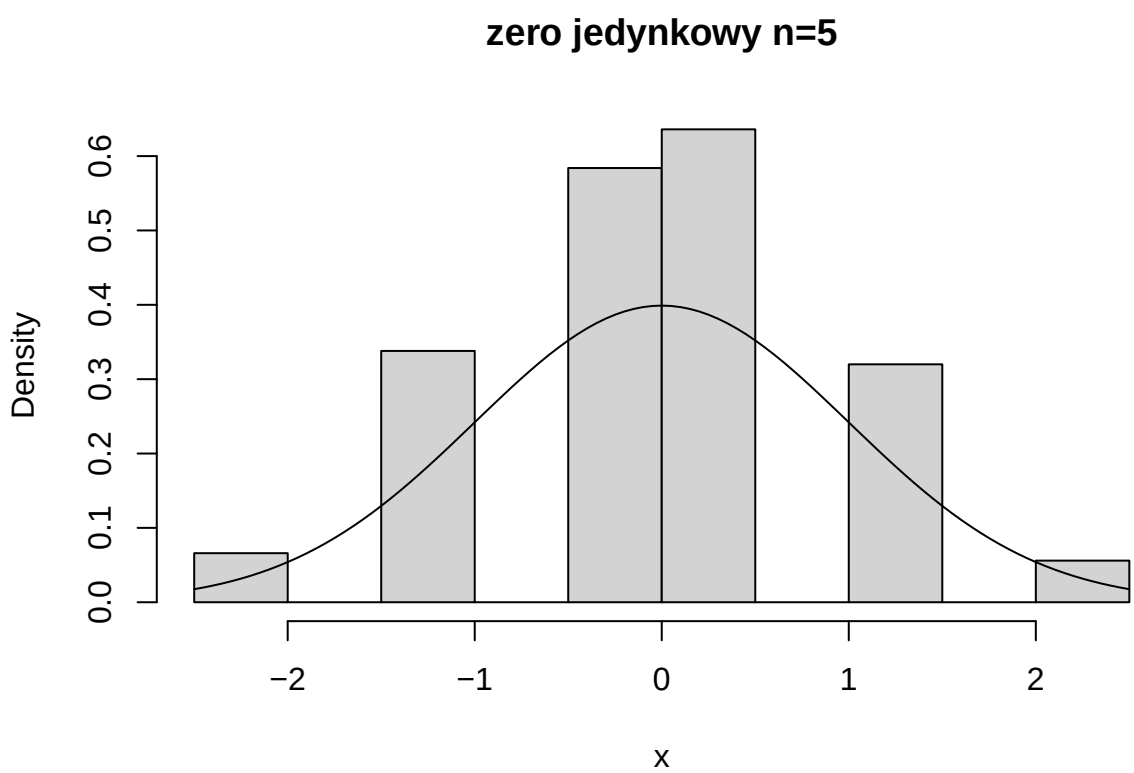
3 sekcja 3

3.1 zadanie 1

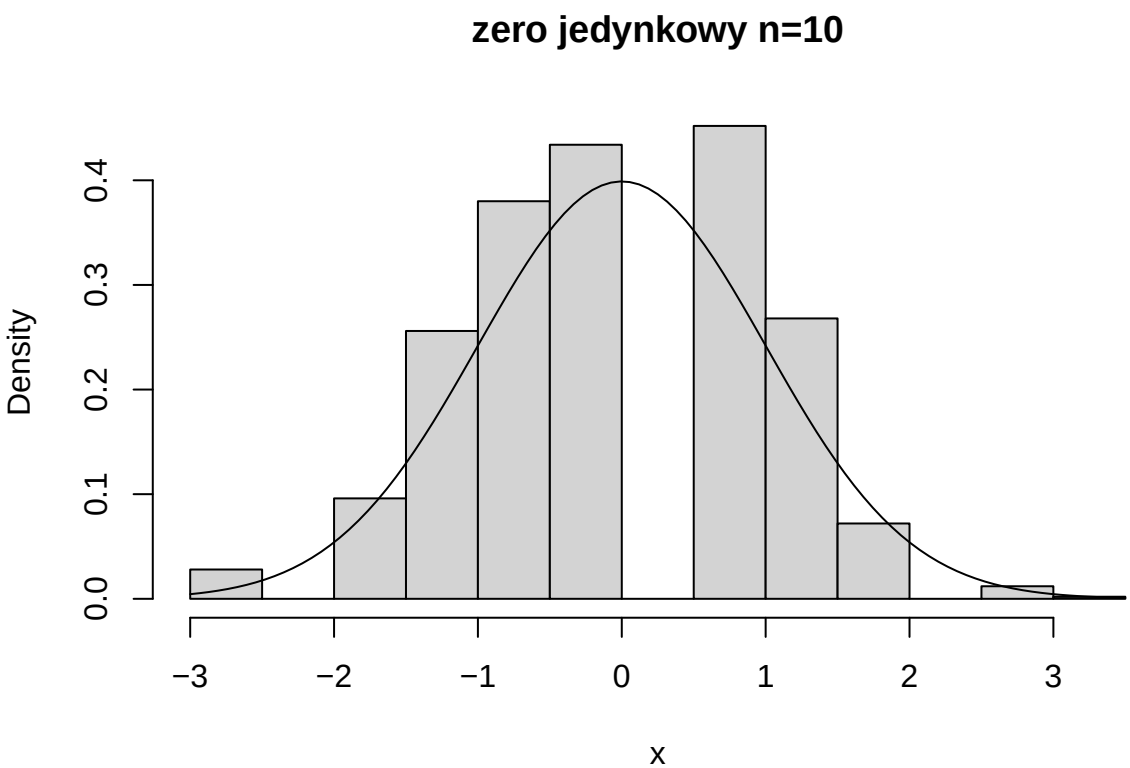
```
n<-1
A<-matrix(sample(c(0,1),n*1000, replace=TRUE), n, 10000)
x<- (colMeans(A) -0.5)/(0.5/sqrt(n))
hist(x, prob=TRUE, main="zero jedynkowy n=1")
curve(dnorm, add=TRUE)
```



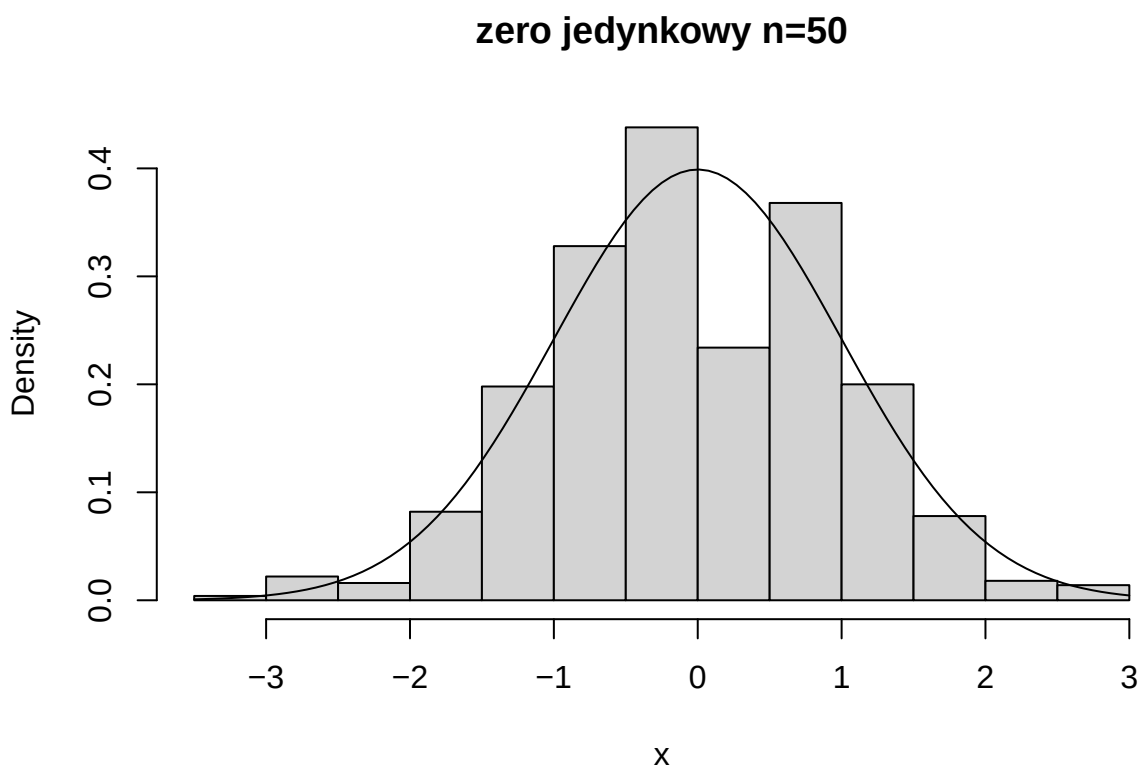

```
n<-5
A<-matrix(sample(c(0,1),n*1000, replace=TRUE), n, 10000)
x<- (colMeans(A) -0.5)/(0.5/sqrt(n))
hist(x, prob=TRUE, main="zero jedynkowy n=5")
curve(dnorm, add=TRUE)
```



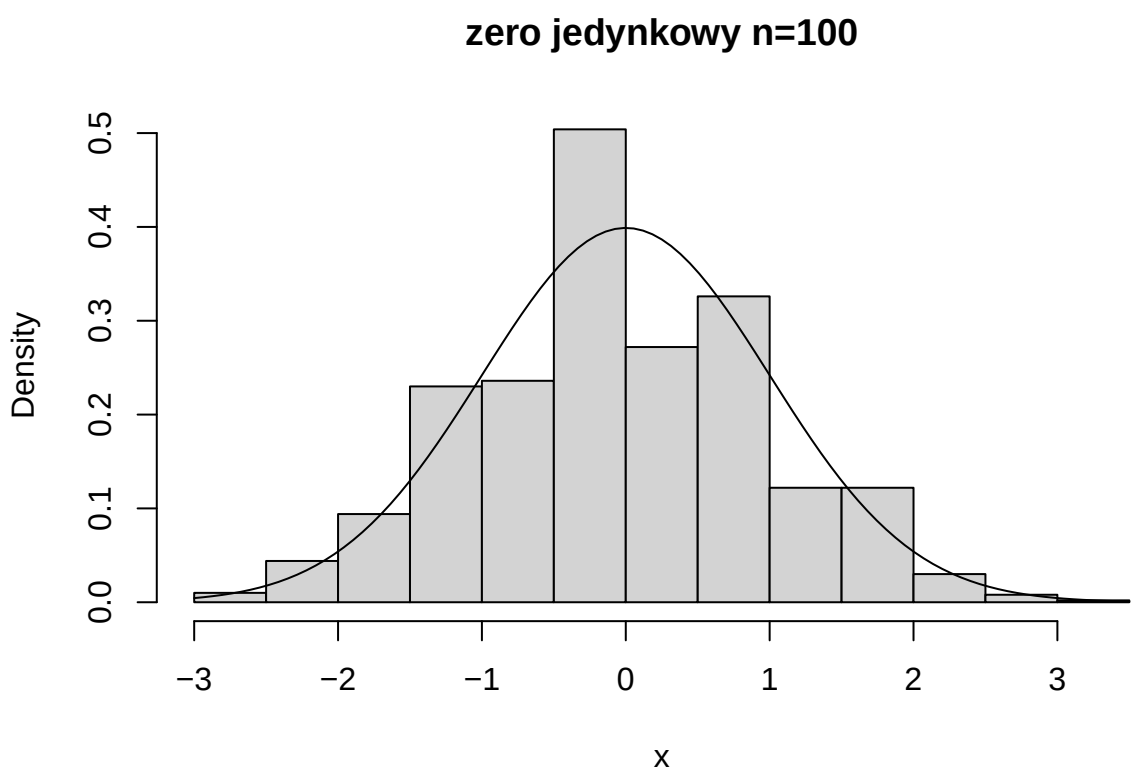
```r
n<-10
A<-matrix(sample(c(0,1),n*1000, replace=TRUE), n, 10000)
x<- (colMeans(A) -0.5)/(0.5/sqrt(n))
hist(x, prob=TRUE, main="zero jedynkowy n=10")
curve(dnorm, add=TRUE)
```



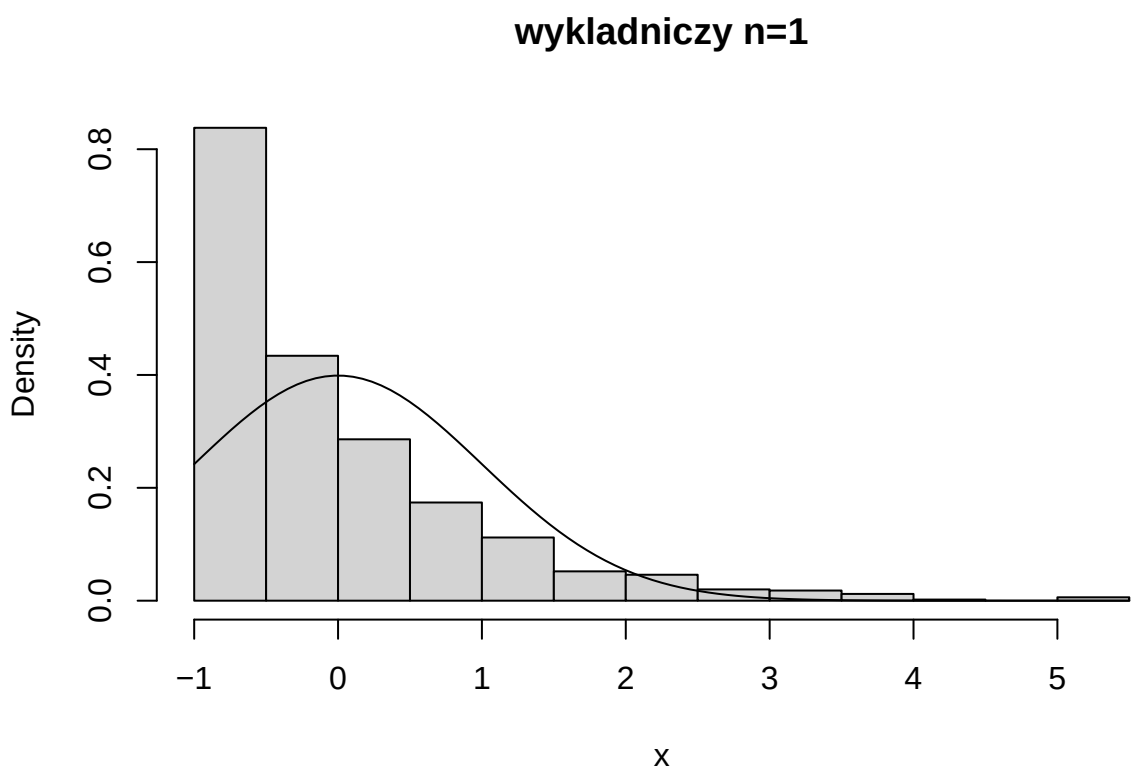

```r
n<-50
A<-matrix(sample(c(0,1),n*1000, replace=TRUE), n, 10000)
x<- (colMeans(A) -0.5)/(0.5/sqrt(n))
hist(x, prob=TRUE, main="zero jedynkowy n=50")
curve(dnorm, add=TRUE)
```



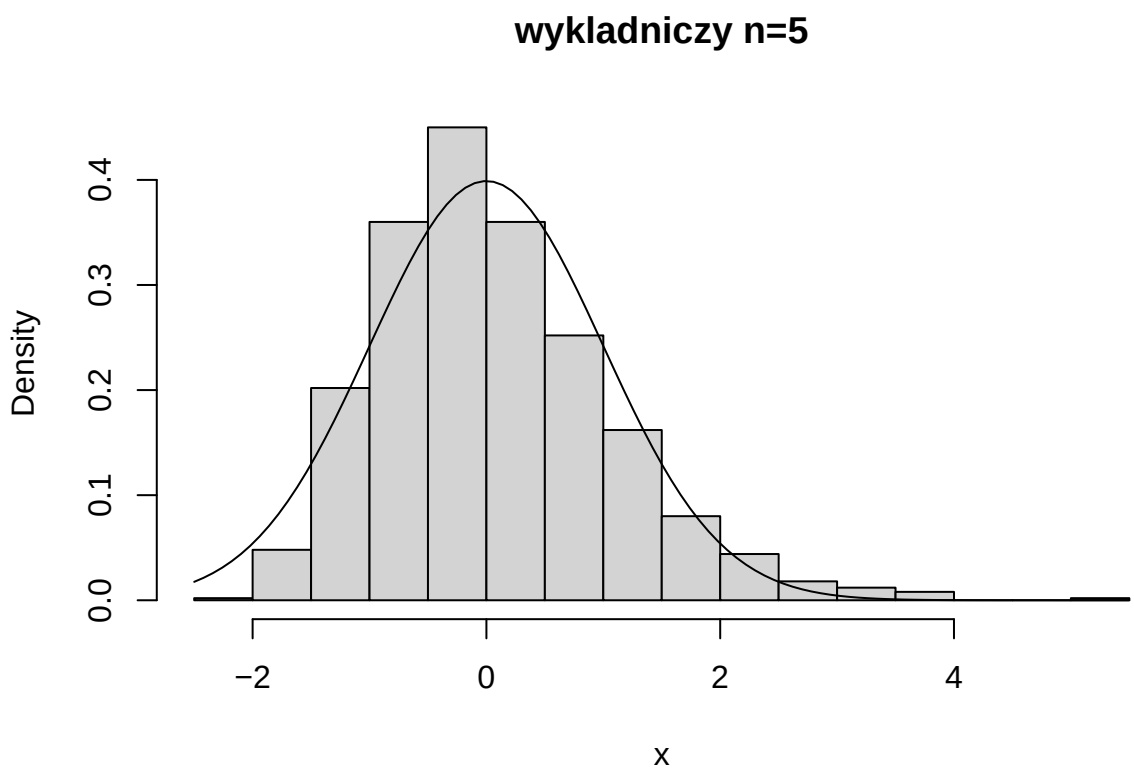

```
n<-100
A<-matrix(sample(c(0,1),n*1000, replace=TRUE), n, 10000)
x<- (colMeans(A) -0.5)/(0.5/sqrt(n))
hist(x, prob=TRUE, main="zero jedynkowy n=100")
curve(dnorm, add=TRUE)
```



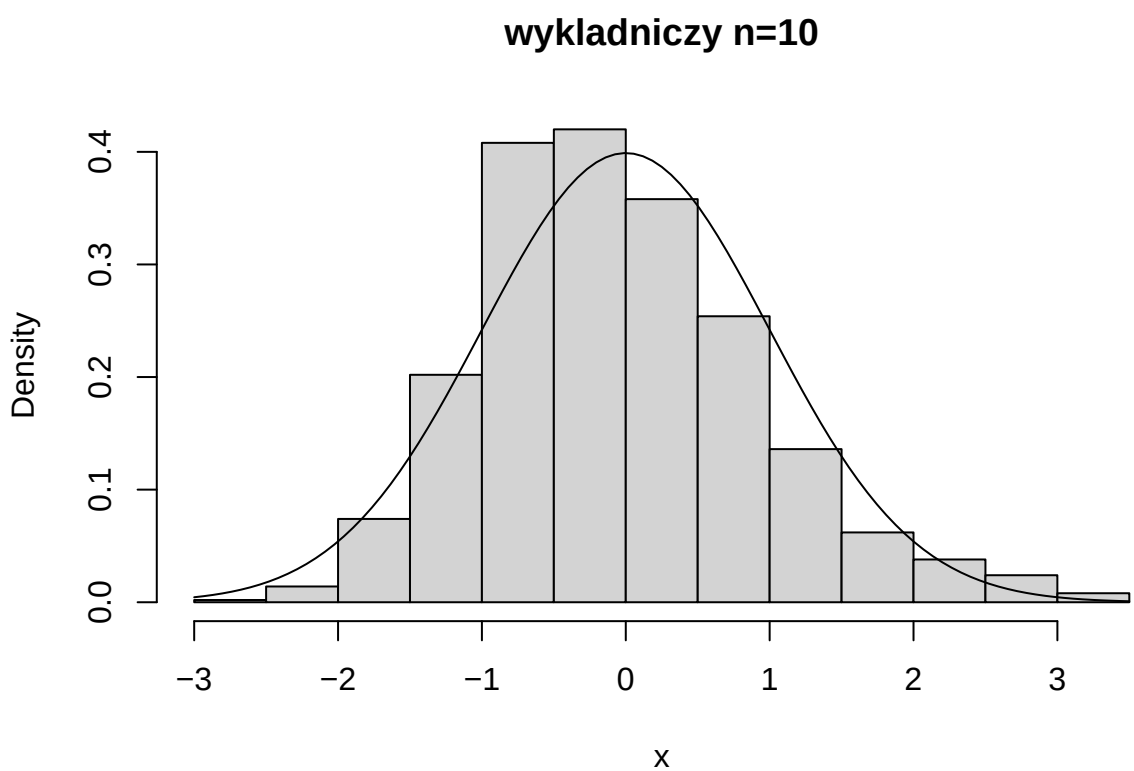
```r
#wykładniczy
n<-1
A<-matrix(rexp(n*1000, 1), n, 10000)
x<- (colMeans(A) -1)/(1/sqrt(n))
hist(x, prob=TRUE, main="wykładniczy n=1")
curve(dnorm, add=TRUE)
```



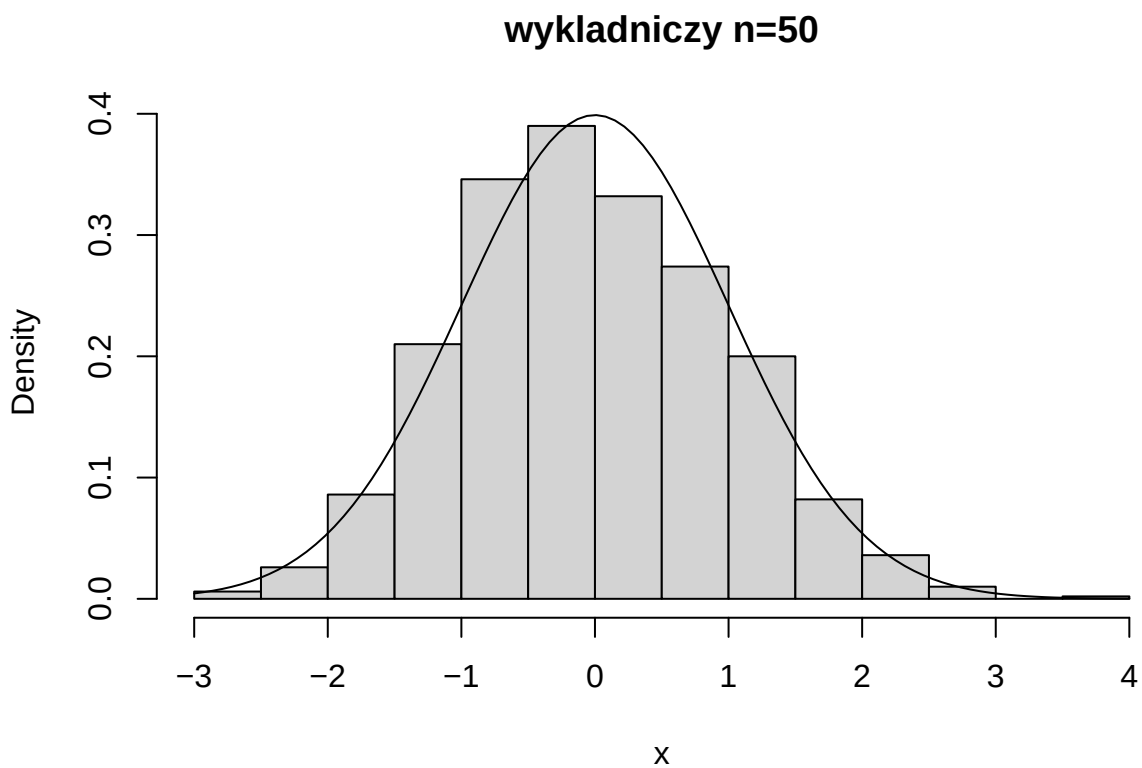

```r
n<-5
A<-matrix(rexp(n*1000, 1), n, 10000)
x<- (colMeans(A) -1)/(1/sqrt(n))
hist(x, prob=TRUE, main="wykładniczy n=5")
curve(dnorm, add=TRUE)
```



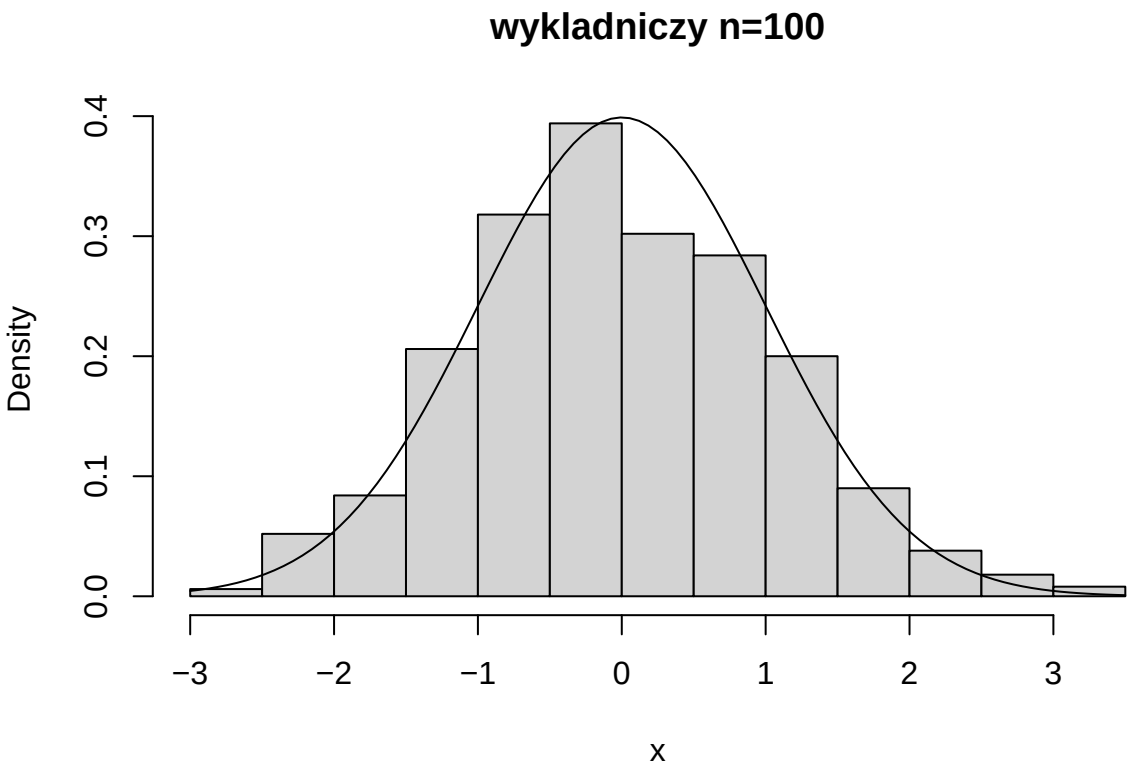

```
n<-10
A<-matrix(rexp(n*1000, 1), n, 10000)
x<- (colMeans(A) -1)/(1/sqrt(n))
hist(x, prob=TRUE, main="wykładniczy n=10")
curve(dnorm, add=TRUE)
```



```r
n<-50
A<-matrix(rexp(n*1000, 1), n, 10000)
x<- (colMeans(A) -1)/(1/sqrt(n))
hist(x, prob=TRUE, main="wykładniczy n=50")
curve(dnorm, add=TRUE)
```




```r
n<-100
A<-matrix(rexp(n*1000, 1), n, 10000)
x<- (colMeans(A) -1)/(1/sqrt(n))
hist(x, prob=TRUE, main="wykładniczy n=100")
curve(dnorm(x), add=TRUE)
```



Dla n=50 możemy zauważyć podobieństwa histogramów do wykresu gęstości rozkładu normlanego. Tak, zależy od wyjściowego rozkładu zmiennych losowych, dla mniejszych n histrogram rozkładu zero jedynkowego w żadnym stopniu nie przypomina gęstości rozkładu normalnego, a wykładniczy natomiast nawet dla małych np. n=10 już przypomina.

3.2 zadanie 2

a)

```
g<-function (f, a,b,n) {
  max<-optimize(f=f,interval=c(a,b), maximum=TRUE)
  f.max<-max$objective
  x<-runif(n,min=a,max=b)
  y<-runif(n, min=0, max=f.max)

  punkty<-sapply(x,FUN=f)

  C<- sum(y<=punkty)
  pole<- (b-a)*f.max
  całka<- pole*(C/n)
  return(całka)

}
```

b)

```
g<-function (f, a,b,n) {
  max<-optimize(f=f,interval=c(a,b), maximum=TRUE)
```

```
  min<-optimize (f=f,interval=c(a,b), maximum=FALSE)
  f.max<-max$objective
  f.min<-min$objective

  przesunięcie<-0

  if (f.min<0) {
    przesunięcie<- -f.min
  }

  x<-runif(n,min=a,max=b)
  y<-runif(n, min=0, max=f.max+przesunięcie)

  punkty<-sapply(x, function (x) f(x)+przesunięcie)

  C<- sum(y<=punkty)
  pole<- (b-a)*(f.max+przesunięcie)
  całka<- pole*(C/n) - (b-a)*przesunięcie
  return(całka)

}
```

c)

```
f<- function(x) x^2+3*x-4

g(f,-4,5,100000)
```

```
## [1] 40.80744
```

```
f<-function(x) sin(x)-6*cos(x-1)

g(f, -10,10,100000)
```

```
## [1] 3.114123
```

```
f<- function(x) x^2+10

g(f, -15, 8, 100000)
```

```
## [1] 1527.661
```

Zwraca poprawne wyniki.

d)

```
f<- function(x) x^2+3*x-4

g(f,-4,5,10)
```

```
## [1] 19.79892
```

```r
g(f,-4,5,100)
```

```
## [1] 38.81115
```

```r
g(f,-4,5,1000)
```

```
## [1] 44.13458
```

```r
g(f,-4,5,10000)
```

```
## [1] 40.06596
```

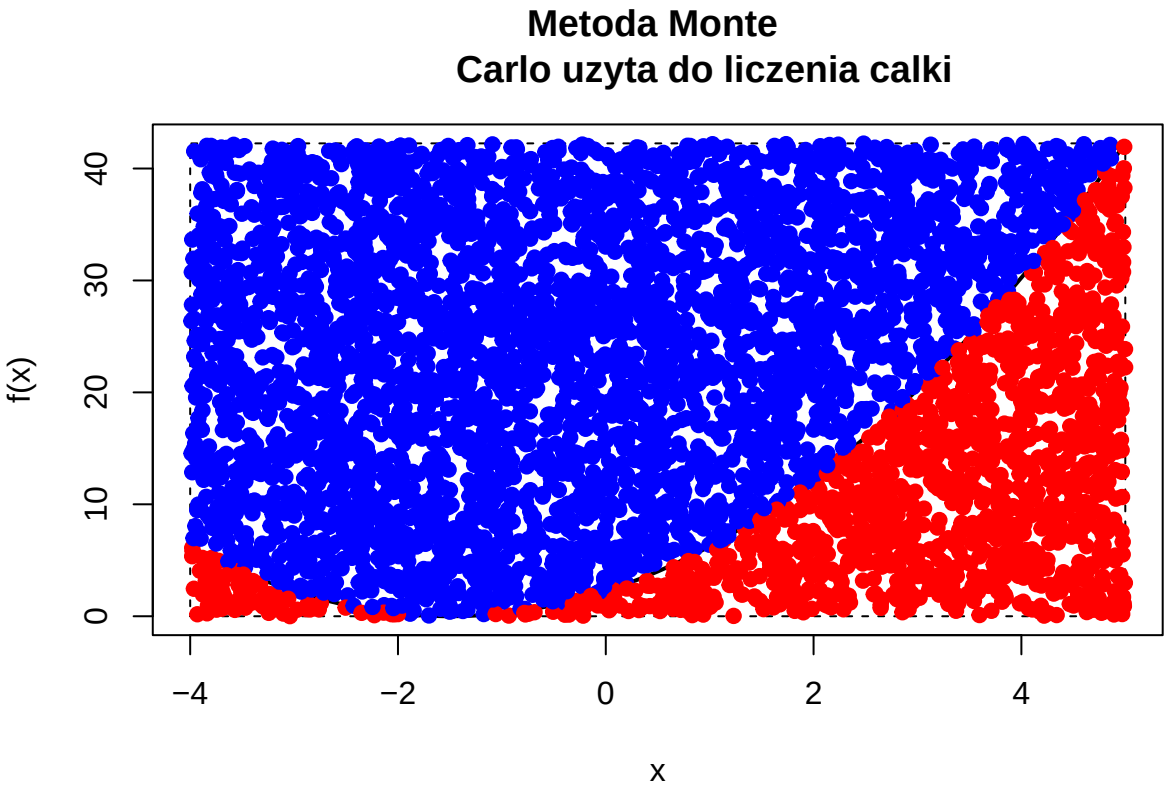
```r
g(f,-4,5,100000)
```

```
## [1] 39.89105
```

Im większe n tym dokładniejszy wynik. Zbieżność jest bardzo wolna potrzeba bardzo dużych n, aby otrzymać zbliżony do prawidłowego wynik.

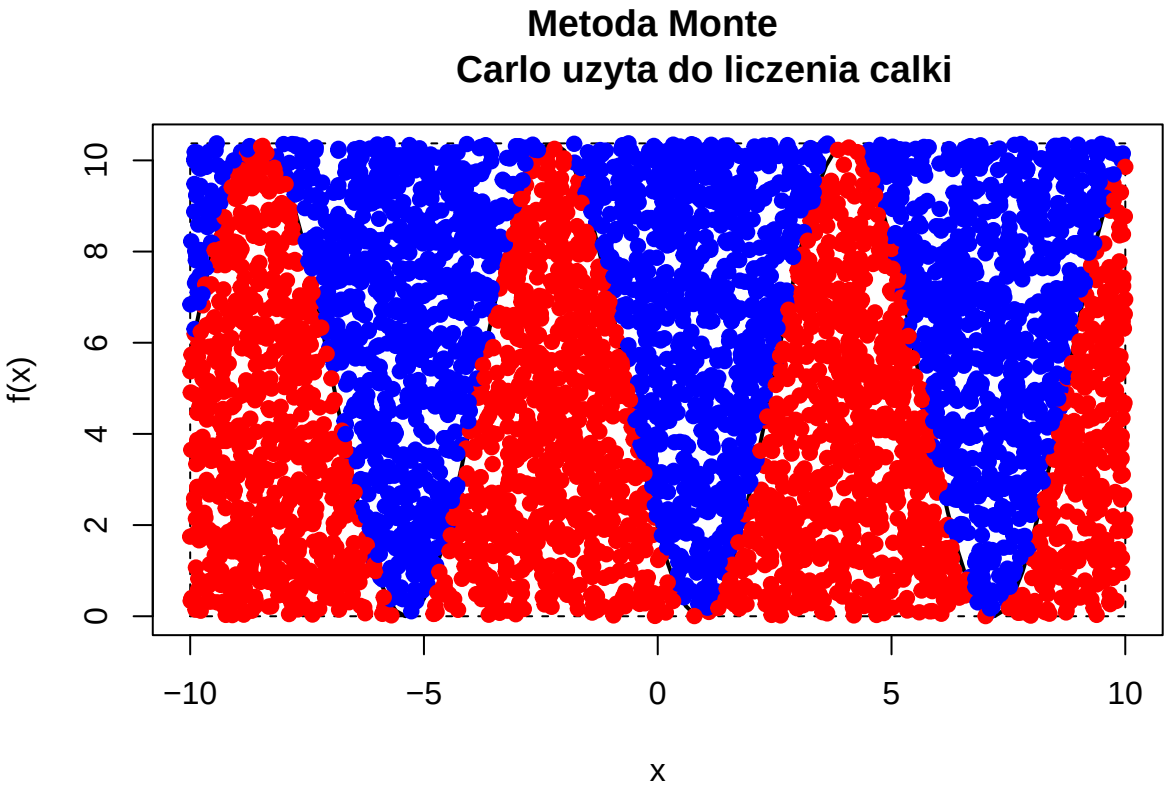
e)

```r
g<-function (f, a,b,n, plot=FALSE) {
  max<-optimize(f=f,interval=c(a,b), maximum=TRUE)
  min<-optimize (f=f,interval=c(a,b), maximum=FALSE)
  f.max<-max$objective
  f.min<-min$objective

  przesunięcie<-ifelse(f.min < 0, -f.min, 0)

  x<-runif(n,min=a,max=b)
  y<-runif(n, min=0, max=f.max+przesunięcie)

  punkty<-sapply(x, function (x) f(x)+przesunięcie)

  C<- sum(y<=punkty)
  pole<- (b-a)*(f.max+przesunięcie)
  całka<- pole*(C/n) - (b-a)*przesunięcie
  if (plot==TRUE) {
    plot(function(x) f(x)+przesunięcie, from=a, to=b, lwd=2,main = "Metoda Monte
         Carlo użyta do liczenia całki", ylab = "f(x)", xlab = "x",
         ylim=c(min(0,f.min+przesunięcie), f.max + przesunięcie))
    rect(a, 0, b, f.max + przesunięcie, lty = 2)
    points(x,y, pch=c(19,30), col=ifelse(y<=punkty,"red", "blue"))
  }
  return(całka)
}

f<- function(x) x^2+3*x-4

g(f,-4,5,10000, plot=TRUE)
```



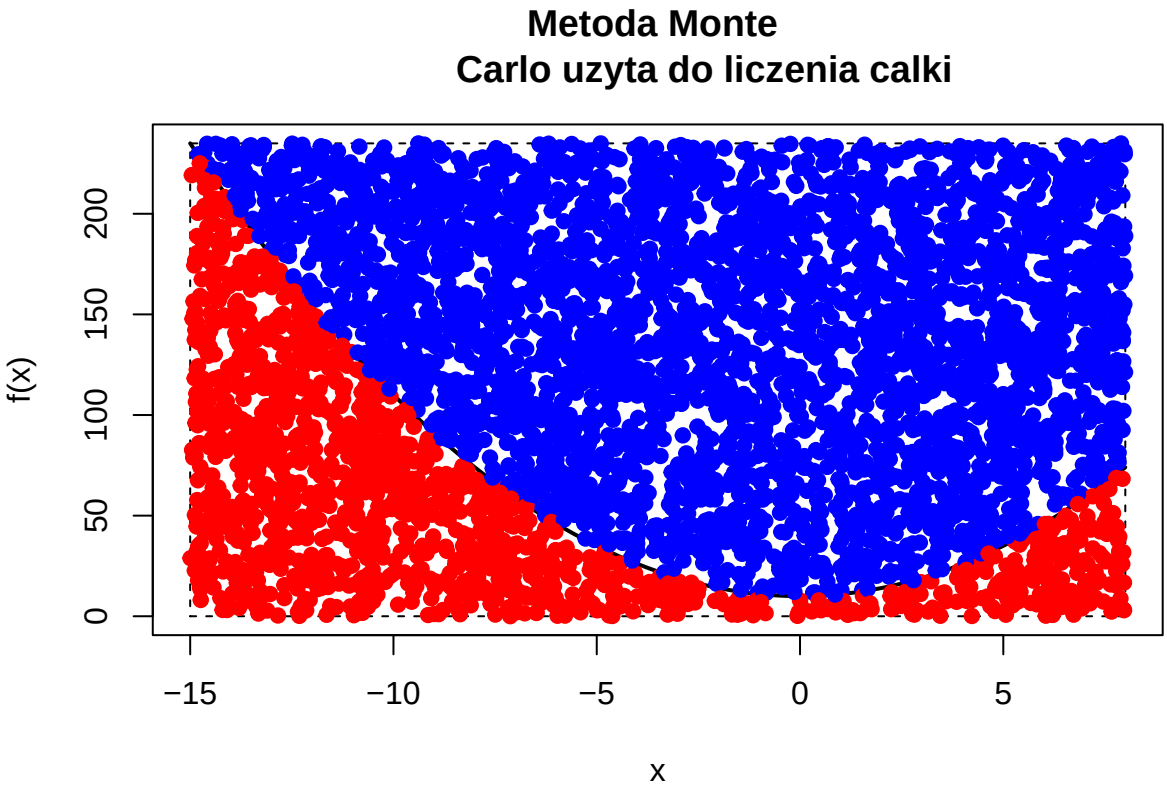

```
## [1] 41.35879
```

```
f<-function(x) sin(x)-6*cos(x-1)

g(f, -10,10,10000, plot=TRUE)
```




```
## [1] 1.639012
```

```
f<- function(x) x^2+10

g(f, -15, 8, 10000, plot=TRUE)
```


Metoda Monte
Carlo uzyta do liczenia calki
f(x)
0
50
100
150
200
−15
−10
−5
0
5
x

```
## [1] 1517.175
```